

Offline coding challenge

Problem: Task Management API

Description:

Build a simplified backend for a **Task Management System** using Java or Golang. The system should allow users to create, update, retrieve, and delete tasks, and each task should have a unique ID, a title, a description, a status, and a due date.

Your solution should be organised using Domain-Driven Design (DDD) principles, with a focus on Test-Driven Development (TDD) and adherence to clean coding practices.

Requirements:

1. **REST API Endpoints:** Implement the following REST endpoints:
 - **Create Task:** `POST /tasks`
 - Request Body: JSON object with `title`, `description`, `status`, and `due_date`.
 - Response: JSON object of the created task, including a unique `id`.
 - Validation: Ensure `title` and `due_date` are mandatory fields.
 - **Get Task:** `GET /tasks/{id}`
 - Response: JSON object of the task with the given `id`.
 - If a task with the specified `id` does not exist, return a `404` status with an appropriate error message.
 - **Update Task:** `PUT /tasks/{id}`
 - Request Body: JSON object with optional updates for `title`, `description`, `status`, and `due_date`.
 - Response: JSON object of the updated task.
 - If a task with the specified `id` does not exist, return a `404` status with an appropriate error message.
 - **Delete Task:** `DELETE /tasks/{id}`
 - Response: Return a `204 No Content` status if successful.
 - If a task with the specified `id` does not exist, return a `404` status with an appropriate error message.
 - **List All Tasks:** `GET /tasks`
 - Response: JSON array of all tasks, sorted by `due_date`.
2. **Task Model:** Each task should have the following fields:
 - `id` (auto-generated unique identifier, string format)
 - `title` (string, required)
 - `description` (string, optional)

- `status` (enum: `PENDING`, `IN_PROGRESS`, `DONE`, default is `PENDING`)
- `due_date` (date, required)
- 3. **Domain-Driven Design:**
 - Separate your code into different layers, such as:
 - **Models/Entities:** Define the `Task` entity.
 - **Repositories:** Responsible for data persistence and retrieval.
 - **Services:** Contains business logic related to task management.
 - **Controllers/Handlers:** Manages HTTP requests and responses.
 - Implement interfaces where possible for better testability.
- 4. **Test-Driven Development:**
 - Write unit tests for each component (models, services, and handlers).
 - Write integration tests for each API endpoint to verify end-to-end functionality.
 - Mock dependencies to isolate components in unit tests.
- 5. **Good Coding Practices:**
 - Follow best practices, including error handling and clean code principles.
 - Use meaningful variable names, proper error handling, and clear structuring.
 - Ensure your code is modular and maintainable.

Constraints:

- Use an in-memory data store or a mock repository for data persistence (no database is required).
- Ensure your code is organised, readable, and well-documented.

Bonus Points:

- Implement pagination and filtering in the **List All Tasks** endpoint, allowing users to filter tasks by status.
- Implement validation for dates to ensure they are valid and in the future.

Submission:

1. Please ensure that the project includes a README file with clear and detailed instructions on how to install the necessary dependencies for compiling and running the code.
2. Submit the code as a ZIP file shared via Google Drive.
3. The deadline for completing this challenge is within two days.